

NPSC3000 End of Year Report: Can Reinforcement Learning Outperform Traditional Methods for Cloud Resource Allocation

Student: Ty Riekert
Supervisor: Elham Mardaneh
Co-supervisor: Tony Mathew

October 2026

Abstract

Resource allocation (RA) is a critical problem for cloud providers [1, 2, 3, 4, 5]: a shared pool of physical servers must serve a continuous stream of user jobs while meeting deadlines and limiting energy use. Classical scheduling methods are known to struggle with the dynamic, multi-objective nature of this problem [6, 7, 8], while reinforcement learning (RL) has shown promise in similar settings [1, 9, 6, 10]. This project asks *under which objectives and conditions RL outperforms classical scheduling methods*. We model cloud RA as a multi-resource scheduling problem in two settings, offline (all jobs known in advance) and online (jobs arrive randomly), and formulate it as a Markov decision process. Six action-space designs were trained with proximal policy optimisation (PPO) and advantage actor-critic (A2C)

under one uniform tuning and training protocol, and compared on 50 held-out instances against 49 dispatching heuristics and the CP-SAT exact solver. On the main objective, weighted squared tardiness, the best RL agent comes within 5% of the best heuristic offline but does not beat it, and only ties the best heuristic when all jobs are equally important. Once linear tardiness or the number of late jobs is added to the objective, RL beats the best heuristic by 2–10% offline. Online, RL needs far more training: with more steps the best design closed most of its gap to the best heuristic but did not overtake it. RL never beat the heuristics when energy was part of the objective. Overall, RL is most competitive when the objective combines several goals that no single hand-designed rule targets, and when jobs differ in importance.

1 Project Aims and Deliverables

The aim of this project is to formulate cloud resource allocation as a mathematical optimisation and reinforcement learning problem, and to rigorously evaluate RL models against established methods. The specific deliverables are:

- Conduct comprehensive research on current findings and machine learning in cloud resource allocation
- Formulate a mathematical model of the cloud RA problem, expressed as a resource-constrained scheduling optimisation problem
- Create a corresponding Markov Decision Process (MDP) formulation for the reinforcement learning agent
- Implement the code of the scheduling environment
- Implement the code for the reinforcement learning agent(s)
- Evaluate the trained agents against classical scheduling heuristics to determine under what conditions reinforcement learning outperforms traditional methods.

The last item is not a single ‘train and compare’ run, but an iterative process repeated throughout the second half of the project: a constant loop of training, evaluating against baselines, investigating behaviour and faults, checking them against the literature, and revising.

2 Background

2.1 Cloud Computing

Cloud computing refers to the use of computing services and resources delivered over the internet, and resource allocation (RA) is one of the most critical challenges facing cloud providers today [1, 11]. As cloud infrastructure underpins the majority of modern computing services, effectively allocating physical resources, like compute or network bandwidth, to a continuously arriving stream of workloads has become of vital importance to providers [2]. Poor allocation leads to poor response time, throughput, makespan, utilisation, energy efficiency and environmental sustainability [12, 13, 5, 14].

2.1.1 Cloud RA

Cloud RA can be decomposed into several coupled problems. Virtual machine (VM) placement decides which physical server hosts each virtual machine, subject to the server’s CPU, memory and other capacities, and is widely regarded as the core RA subproblem [1]. Task scheduling decides when and on which resource each user task runs; depending on the setting it may prioritise execution time, response time, deadlines or an even spread of work [6]. Autoscaling dynamically adjusts resource capacity to match fluctuating demand [9], and

load balancing distributes incoming tasks evenly to avoid hotspots [15]. This project studies the task-scheduling view: deciding when and where each job runs.

Solutions to these problems generally fall into three categories:

- **Exact methods.** Algorithms, such as integer and constraint programming, that are guaranteed to find an optimal solution given enough time.
- **Approximation methods.** Used when exact methods become computationally infeasible. *Heuristics* are problem-specific rules, designed by people from domain knowledge, that quickly construct or improve a single solution (for example, “always run the job with the earliest deadline”). *Metaheuristics* are general-purpose search strategies, such as particle swarm optimisation, that guide heuristic methods through the space of solutions [16].
- **Machine learning (ML).** Methods that learn a decision rule from data or experience instead of having it written by hand. For scheduling, a model is trained on many example problems, and the trained model then makes allocation decisions on new problems.

Exact methods guarantee optimality, but scheduling problems of this kind are NP-hard [17], so their run time grows rapidly with problem size; in practice they are limited to small instances, or are stopped early and used for the best solution and bounds they have found. Heuristics and metaheuristics scale well but give approximate solutions with no guarantee of quality. Systematic reviews identify four recurring limitations of these approximation methods in cloud RA: scalability to large systems, adaptability to changing workloads, handling several conflicting objectives at once, and the need for real-time decisions [5, 6, 7, 8, 18, 19].

2.1.2 Motivations for Machine Learning

These limitations motivate a growing body of work applying machine learning, and reinforcement learning specifically, to combinatorial and scheduling problems. A fixed heuristic relies on its designer’s intuition about what makes a job important, and applies that rule unchanged in deployment. RL still requires design choices, such as what to optimise, but the rule for assigning jobs to machines is learnt during training, in an environment that should reflect the deployment environment [18]. Systematic reviews of RL for scheduling and bin packing report encouraging growth in this area [8], and prior systems such as DeepRM [20] and Decima [21] showed learned schedulers matching or exceeding hand-designed rules on cluster workloads.

This project examines two actor-critic RL methods, Proximal Policy Optimisation (PPO) and Advantage Actor-Critic (A2C), the synchronous version of A3C [22, 23, 24, 25]. It also examines RL as a selection hyper-heuristic, where the agent chooses between dispatching rules [26, 27, 28], and hybrid designs in which RL learns job priorities while a fixed rule places jobs on machines [29, 30]. These are compared against classical dispatching heuristics, a metaheuristic and an exact solver [31, 32].

2.1.3 Project Motivation

RL for cloud RA is an active research area [18, 8], but existing papers mostly optimise only a few objectives and compare against a limited set of baselines. Most model the problem as bin packing or VM placement, and many focus on resource usage alone [33]; a multi-objective, resource-constrained scheduling view is less explored. Building on the problem structures, models and findings of earlier work, we implement a suite of RL approaches and compare them against a broad set of classical methods, to see how well RL really performs. Our research question is:

Under which objectives and conditions does RL outperform classical scheduling algorithms for cloud RA?

2.1.4 Markov Decision Process

The Markov Decision Process (MDP) is a foundational framework for decision making in stochastic and uncertain environments [34], and is the core structure of an RL problem. At each time t the environment is in a state s_t ; the agent chooses an action a_t from the actions available in that state (infeasible actions are often masked out [35]), and the environment then moves to a new state s_{t+1} .

An MDP is described by a tuple (S, A, P, R, γ) , where S is the state space and A the action space. P gives the transition probabilities, which satisfy the Markov property

: the next state depends only on the current state and action,

$$p(s_{t+1} \mid s_1, a_1, \dots, s_t, a_t) = p(s_{t+1} \mid s_t, a_t). \quad (1)$$

$p(s' \mid s, a)$ is the probability of moving to state s' after taking action a in state s . The reward function $R(s, a) = \mathbb{E}[r_t \mid s_t = s, a_t = a]$ gives the expected immediate reward for taking action a in state s . Finally, the discount factor $\gamma \in (0, 1]$ sets how much future rewards count relative to immediate ones. The agent's goal is to maximise the total (discounted) reward. The MDP loop is shown in Fig. 1.

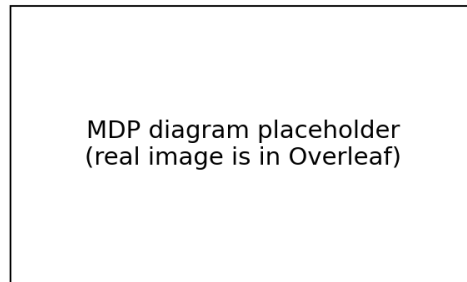


Figure 1: Agent interaction with an MDP.

2.2 Reinforcement Learning

In RL, an agent with no prior knowledge learns to make decisions from repeated interaction with an environment

, using the reward it receives as feedback [36]. The problem is formalised as an MDP (Section 2.1.4) [36, 34], and the agent learns a *policy*: a probability distribution over actions a_t given the state s_t , chosen to maximise the expected total reward. RL methods fall into two main families. **Value-based** methods learn the expected return of each action and typically choose the action with the highest estimate, while **policy-gradient** methods directly adjust the policy’s parameters towards higher return [37]. **Actor-critic methods** combine the two: a critic learns the expected return from each state (a value function), and an actor updates the policy using the critic’s

estimates to judge whether an action was better or worse than expected [24]. PPO and A2C are both actor-critic methods.

3 Methodology

3.1 Problem Formulation

A cloud computing environment receives a set of jobs that must be assigned to a set of physical machines. Each job j has a fixed processing duration p_j , a multi-dimensional vector of resource requirements a_{jr} (e.g. CPU, memory) which it occupies for its entire duration, and a due date d_j by which it should finish. Jobs are independent of one another. Each job has a priority weight w_j , assumed to be set externally by business needs, to reflect that some jobs matter more than others.

Jobs are served by a set of identical machines M , each with a fixed capacity C_r for every resource r . A machine may process several jobs at once, provided their combined usage does not exceed its capacity in any resource. Scheduling is non-preemptive: each job is assigned to exactly one machine at a single start time s_j and runs uninterrupted until completion. Time is divided into integer steps over a planning horizon H . The tardiness of job j is

$$T_j = \max(0, s_j + p_j - d_j), \quad (2)$$

and $U_j = 1$ if job j finishes late ($T_j > 0$), 0 otherwise. Notation is summarised in Appendix A and the full constraint set in Appendix B.

3.2 Objective Function

The first objective tried weighted the number of machines used, weighted tardiness and peak utilisation θ

:

$$\min \lambda_1 \sum_{m \in M} y_m + \lambda_2 \sum_{j \in J} w_j T_j + \lambda_3 \theta. \quad (3)$$

Equation (3) could be gamed: unscheduled jobs cost nothing, so agents found high-reward schedules that left many jobs unassigned, a form of reward hacking [38]. Its peak-utilisation term also works against energy-efficient consolidation [39, 40].

It was replaced by a weighted sum of optional terms:

$$\min J_\lambda = \underbrace{\sum_j w_j T_j^2}_J + \lambda_T \sum_j w_j T_j + \lambda_U \sum_j w_j U_j + \lambda_E E, \quad (4)$$

covering squared tardiness, linear tardiness, the weighted number of late jobs (deadline violations), and energy E , measured as total active machine-time [40]. The main objective of this project is J alone ($\lambda_T = \lambda_U = \lambda_E = 0$). Squared tardiness penalises a few very late jobs more than many slightly late ones, whereas linear tardiness can score very different schedules equally. J is evaluated over an *extended horizon*: jobs are released only within $[0, H)$, but no job is ever discarded, and an unfinished job keeps accumulating tardiness until it completes. Leaving a job unscheduled is therefore always costly, which removes the loophole in Eq. (3). The other terms are switched on only in the multi-objective experiments (Section 3.7).

3.3 Offline and Online Cases

Two cases of the problem are studied. In the **offline** case all jobs are known and available at $t = 0$, and at most one job can be started per time step. In the **online** case jobs arrive (are revealed) over time according to a Poisson process [41, 20], and any number of jobs may be started at the same time step.

The difficulty of the online case is set by the load factor

ρ , which measures how busy the cluster is on average. A job j brings $p_j a_{jr}$ units of work for resource r (resource units multiplied by time steps). If jobs arrive at rate ν per time step, on average $\nu \mathbb{E}[p_j a_{jr}]$ units of work arrive per time step, while the cluster can process at most $|M| C_r$ units per time step. Their ratio is the standard utilisation of a queueing system [41], and taking the largest ratio over resources picks out the bottleneck resource:

$$\rho = \max_{r \in R} \frac{\nu \mathbb{E}[p_j a_{jr}]}{|M| C_r}. \quad (5)$$

For $\rho < 1$ the cluster can keep up on average; for $\rho \geq 1$ work arrives faster than it can be processed and the queue necessarily grows. For each target load, the arrival rate ν is chosen to give that ρ . Loads from $\rho = 0.50$ to 1.10 are studied.

3.4 Markov Decision Process

To formulate this problem for our algorithms we use an MDP (S, A, P, R, γ) as described in Section 2.1.4.

State S . The state holds the features $F_j = (p_j, a_j, d_j, w_j)$ of each waiting job $j \in J_t \subseteq J$, and, for each job in the processing set $P_t \subseteq J$ (jobs that have started and not finished), $G_j = (m_j, s_j, p_j)$: its machine, start time and duration. It also holds the remaining capacity of each machine,

$$R_{mrt} = C_r - \sum_{j \in P_t: m_j=m} a_{jr}, \quad (6)$$

and $y = (y_m)_{m \in M}$, recording which machines are switched on. This gives the state

$$s_t = (J_t, P_t, \{F_j\}_{j \in J_t}, \{G_j\}_{j \in P_t}, \{R_{mrt}\}_{m \in M, r \in R}, y, t). \quad (7)$$

The agent also sees each machine’s free capacity over the next few time steps, computed from G_j (no new information, but easier to learn from).

Action space A . At each step, the agent can

assign a waiting job to a machine, or idle:

$$A_t = \{(j, m) : j \in J_t, m \in M\} \cup \{\text{idle}\}. \quad (8)$$

Assignments that would exceed a machine’s capacity are masked out [35]. Offline, each assignment is followed by a time step; online, the agent may make several assignments per time step, and time advances only when it idles. Idling is always allowed: deliberately leaving capacity free for later jobs is a behaviour RL could learn, and schedules that never idle can exclude the optimum [42]. An idle advances time to the next job arrival or completion, so the agent cannot idle forever.

Transition dynamics P . An assignment (j, m) moves job j from J_t to P_t with $s_j = t$ and $m_j = m$, switching machine m on if needed. When time advances, completed jobs leave the processing set by Eq. (9) and release their capacity; online, newly arrived jobs join J_{t+1} . Transitions are deterministic offline, and random online only through

the arrivals.

$$P_{t+1} = \{k \in P_t : s_k + p_k > t + 1\}. \quad (9)$$

Reward R . The reward is the negative of the objective, charged as it accrues: a late job of weight w_j is charged $w_j(2k + 1)$ in its k -th late time step, and since $\sum_{k=0}^{T_j-1} (2k + 1) = T_j^2$ the charges add up to exactly $w_j T_j^2$. Maximising total reward is therefore the same as minimising J in Eq. (4). To give the agent earlier feedback, a potential-based shaping term is added, which provably leaves the optimal policy unchanged [43].

Action-space designs. The full action space Eq. (8) has 1001 actions for 100 jobs and 10 machines. Large discrete action spaces are a known difficulty for policy-gradient methods [44], while successful prior schedulers use far fewer actions [20, 21]. Six designs were therefore compared:

- **Option 0 – Direct job-machine selection.** The policy picks a (job, machine) pair from Eq. (8), using a pointer-network architecture [45, 46].
- **Option 1 – Rule selection.** At each decision the policy chooses which dispatching rule to apply: one of seven priority rules, each with First-Fit or Consolidate placement (Section 3.5), making the agent a selection hyper-heuristic [26, 27].
- **Option 2 – Priority learning.** The policy picks which waiting job to start next, learning its own priority from the raw job features; the chosen job is then placed by the First-Fit rule. This is a hybrid of RL and a dispatching heuristic: RL learns the ordering, a fixed rule does the placement [20, 29].
- **Option 3 – Priority learning with an ATC feature.** Option 2 with the Apparent Tardiness Cost index [31] added as an extra job feature. Option 3w is the same, restricted to the 20 most urgent jobs [20].
- **Option 4 – Action branching.** The job and the machine are chosen by two separate network heads instead of one joint action [47].

3.4 Learning Algorithms

All RL agents use PPO [22] or A2C [23], as implemented in Stable-Baselines3 [48]. Both are actor-critic methods (Section 2.2). PPO limits how far the policy can change in a single update, which makes training more stable; A2C uses simpler, more frequent updates. Both are widely used and comparable within one framework [49].

Approaches tried before the final setup. The final setup was reached through several rounds of training, diagnosis and revision. Alternatives tried along the way included reward-constrained policy optimisation (RCPO) [50] and PPO-Lagrangian [51], which treat tardiness as a constraint rather than a cost, dense and shaped tardiness rewards, curriculum learning, and a version in which idling was forbidden whenever a job could be placed. Under the original objective (Eq. (3)), none of these produced an agent that beat the best heuristic on tardiness. The final version fixed two faults in the previous one: the critic never learned, because squared-tardiness rewards are very large, and a deterministic policy could idle forever. These were fixed by reward scaling [52, 53], event-driven idling (Section 3.4) and a two-stage evaluation rule (Section 3.7). In addition, the critic (but not the policy) sees a summary of jobs that have not arrived yet, which reduces noise in online training without biasing the policy [54, 55]. Appendix D gives the diagnosis.

3.5 Methods Compared

Dispatching heuristics. Each heuristic combines a priority rule, which chooses the next job, with a placement rule, which chooses its machine [56].

- **Priority rules:** Earliest Deadline First (EDF) and Least Slack Time (LST), which prioritise urgency [56]; Shortest and Longest Processing Time (SPT, LPT) [56, 57]; Weighted SPT (WSPT) [58]; First-Come-First-Served (FCFS, also known as First-In-First-Out, FIFO); Apparent Tardiness Cost (ATC), which balances job weight, length and urgency [31]; and the weighted-tardiness rules WMDD [59] and COVERT [60]. Because the published weighted rules performed poorly under squared tardiness, three weighted variants were constructed for this project: weighted LST (WLST) and weighted EDF (WEDF), which scale a job’s slack by its weight, and Marginal Delay Cost (MDC), which applies Smith’s ratio rule [58] to the marginal increase in squared tardiness. These were added after the first RL results were seen.
- **Placement rules:** First-Fit, Best-Fit and Worst-Fit [61], and Consolidate, an energy-aware rule that prefers machines that are already on [39].
- **Tetris:** a multi-resource packing heuristic that scores job-machine pairs jointly [62].

In total, the twelve priority rules and four placement rules give 48 heuristics, plus Tetris; random rule choice is included as a sanity check.

Metaheuristic. Particle Swarm Optimisation (PSO) [63], in which each particle encodes a job ordering that a placement rule decodes into a schedule, was evaluated in earlier rounds of the project. It was worse than the best dispatching rule in every setting tested, so it is not part of the final comparison.

Exact solver. CP-SAT [32], a constraint-programming solver, models the problem of Appendix B exactly. Within a time limit it returns the best schedule it has found, whose J is an upper bound on the optimum, together with a proven lower bound on the optimum. When the two meet, the schedule is proven optimal. CP-SAT was given 60 seconds per instance.

3.6 Implementation

The environment, agents and evaluation pipeline are implemented in Python¹ using Gymnasium and Stable-Baselines3 [48]. The environment and the translation of the mathematical formulation into code were written by hand; model implementation, training and evaluation were done with the help of Claude Code (

¹<https://github.com/Ethan-Ty-Riekert/Neural-Knapsack-Research-Project>

see the Generative AI Use Statement).

3.7 Experimental Setup

Instances. Offline instances have 100 jobs, 10 machines and 4 resources with $C_r = 30$, and $H = 100$. Job durations and resource demands are drawn uniformly from

$\{1, \dots, 9\}$, and weights from $\{1, \dots, 5\}$. Deadline tightness is set by the tardiness factor TF [64]: larger TF means tighter deadlines on average. The main setting uses $TF = 0.5$ (medium), with 0.2 and 0.8 as loose and tight variants. A unit-weight setting ($w_j = 1$) isolates the effect of job weights. Online instances always use weighted jobs (there is no unit-weight online setting) and add Poisson arrivals at loads $\rho \in \{0.50, 0.75, 0.95, 1.10\}$; each job’s deadline is its arrival time plus a random slack, the online counterpart of TF. Online job sizes are log-normal (right-skewed, like real cluster traces [65, 66]). Because every RL study takes hours to days of compute, RL agents were trained and evaluated in one offline setting ($TF = 0.5$, with weighted and unit weights) and one online setting ($\rho = 0.95$: heavy load, but below saturation, so scheduling decisions matter most). The other loads and deadline tightnesses were run for the heuristics only. Appendix D gives the exact distributions.

Training and tuning. An agent learns over episodes, each a complete, newly generated problem instance, so it never trains on the same problem twice. Training, validation (20 instances) and test (50 instances) use disjoint random seeds, so test instances are never seen in training or tuning. Every combination of design (6), algorithm (2) and case (2) had its own hyperparameter search, 24 studies in all, run with the same protocol: Optuna [67] with the TPE sampler [68], 12 trials of 300,000 steps, the first trial always being the library defaults. Poor trials were stopped early by median pruning, which compares a trial’s validation J with the median of earlier trials *at the same training step*. Each study keeps one best configuration, chosen on the 20 validation instances. That configuration was then retrained with 3 random seeds for 1,000,000 steps offline and 300,000 steps online, and results are the mean and standard deviation over the seeds. A follow-up trained online Options 0, 2 and 3 (PPO) for 1,000,000 steps, and Option 2 for 2,000,000. Appendices C and D list the search space and protocol details.

Evaluation. Every method, RL and baseline, is evaluated on the same 50 held-out test instances for each setting. RL agents act deterministically using a two-stage rule: idle if the probability of idling is at least the total probability of all job actions, otherwise take the most likely job action. A plain “most likely action” rule is biased towards idling (Section 3.4

). We report mean J over the test instances; for RL this is averaged over the 3 seeds, with the standard deviation across seeds. The heuristics are deterministic, so they were run once; their standard deviation is across instances.

We also record metrics that were not optimised, such as the on-time rate, number of late jobs, maximum tardiness and energy. Any pattern in these is a side effect of optimising J rather than a learned trade-off, so it is interpreted with caution. Some metrics are uninformative by construction: offline, exactly one job starts per time step, so the mean waiting time is 49.5 for every method.

Multi-objective experiments. To test whether RL’s relative performance depends on the objective, every method was also scored on composite objectives J_λ (Eq. (4)). The weights $\lambda_T, \lambda_U, \lambda_E$ are set separately for each setting, so that each added term equals J on the best heuristic’s schedule; this prevents any term from dominating merely because of its units. Each weight is then multiplied by 0.25, 0.5, 1, 2 and 4. In addition to the agents trained on J alone, Option 2 and 3 PPO agents were trained directly on several composite objectives (300,000 steps offline with default hyperparameters, 3 seeds; 1,000,000 steps online).

4 Results

All results below are on the 50 held-out test instances; lower J is better. RL results are for the three settings RL was trained on: offline with weighted jobs, offline with unit weights, and online (weighted jobs) at $\rho = 0.95$. The other loads and deadline settings appear only in the heuristic comparison (Section 4.3). Full tables for every setting and metric are on the project’s GitHub results page.

4.0 RL against heuristics on the main objective

Table 1 and Fig. 2 summarise the main comparison. Offline, with weighted jobs, the best RL agent (Option 2, PPO) reaches $J = 28,061$, 5.2% above the best heuristic, WLST (26,670). RL’s standard deviation across seeds is small (219), so the gap is not an artefact of one unlucky seed. Every design except Option 1 lands within 12% of WLST, and all of them beat every published rule: the best of these, LST, scores 39,536, 29% worse than the best RL agent. The weighted rules WLST, WEDF and MDC were designed for this project after RL had beaten LST, and only WLST beats RL. CP-SAT’s best schedule after 60 seconds (35,681) is worse than both RL and WLST, so it gives no useful bound on how far either is from optimal at this size. On 15-job instances, however, CP-SAT proved 13 of 15 instances optimal, and LST matched the optimum on every one.

When every job has the same weight, the problem is easier for the heuristics: LST scores 13,098, and the best RL agents either reproduce it exactly (Option 1) or come within 0.8% (Option 2).

Online at $\rho = 0.95$, under the standard protocol (300,000 steps), every RL design is far behind the best heuristic, EDF+Consolidate (21,986): the best is Option 1 A2C at 37,812

(+72%). The online gap is mainly a training-budget problem (Section 4.2).

Table 1: Mean J on 50 test instances (lower is better). RL: mean $\pm$ standard deviation over 3 seeds, best algorithm per design; heuristics are deterministic. Online RL is shown at the standard 300k-step budget; longer budgets are in Table 2.

Method	Offline, weighted	Offline, $w_j = 1$	Online, $\rho = 0.95$
Best heuristic	26,670 (WLST)	13,098 (LST)	21,986 (EDF+Cons.)
LST+FirstFit	39,536	13,098	38,831
EDF+FirstFit	41,302	13,757	40,381
CP-SAT (60 s)	35,681	16,469	–
Option 0 (job-machine pairs)	29,696 $\pm$ 833	13,494 $\pm$ 276	44,920 $\pm$ 0
Option 1 (rule selection)	38,645 $\pm$ 142	13,098 $\pm$ 0	37,812 $\pm$ 8,893
Option 2 (priority)	28,061 $\pm$ 219	13,207 $\pm$ 28	39,947 $\pm$ 11,949
Option 3 (priority + ATC)	28,282 $\pm$ 537	13,496 $\pm$ 98	44,827 $\pm$ 2,890
Option 4 (action branching)	28,577 $\pm$ 43	13,411 $\pm$ 181	139,721 $\pm$ 101,396
Best RL vs best heuristic	+5.2%	0.0%	+72%

4.1 Combined objectives

Figure 3 answers the research question in one view: for each objective and setting, it compares the best RL agent with the best heuristic. Offline with weighted jobs, RL overtakes the best heuristic once the objective rewards more than squared tardiness. Once linear tardiness counts at least as much as the reference ($\times 1$), RL is better by 1.8–10.3%; with late jobs added at weights $\times 0.5$ to $\times 4$, by 1.4–10.0%; and with all four terms, by 8.6%. At moderate weights the winning agents were trained on J alone; at the largest weights, agents trained on the combined objective win. Energy is the exception: RL loses by 3.4–4.8% at every weight.

This advantage depends on job weights and on the setting. With unit weights, RL never beats LST (0.0% to +10.8%). Online, RL trails the best heuristic by 5.8–15.7% on every combined objective.

Figure 4 shows why RL wins offline. WLST reaches the lowest J by accepting many late jobs (a weighted count of 183). Every heuristic lies on or above the dashed line of the best trade-offs any heuristic achieves, but most RL agents lie below it: for a similar J , they finish noticeably more jobs on time. Agents trained on a combined objective (green) move further towards fewer late jobs.

4.2 Online training budget

More training closes most of the online gap for Option 2 but not for Option 0 (Table 2). Option 2’s mean J falls from 39,947 at 300k steps to 26,515 at 1M (+21% over EDF+Consolidate) and 24,921 at 2M (+13%, 2 seeds), and its best 1M seed is within 4.3% of the heuristic. Option 0 improves on average but remains 55% behind at 1M. One Option 2 agent trained for 1M steps on J plus linear tardiness matched EDF+Consolidate (22,000 vs 21,986), but a

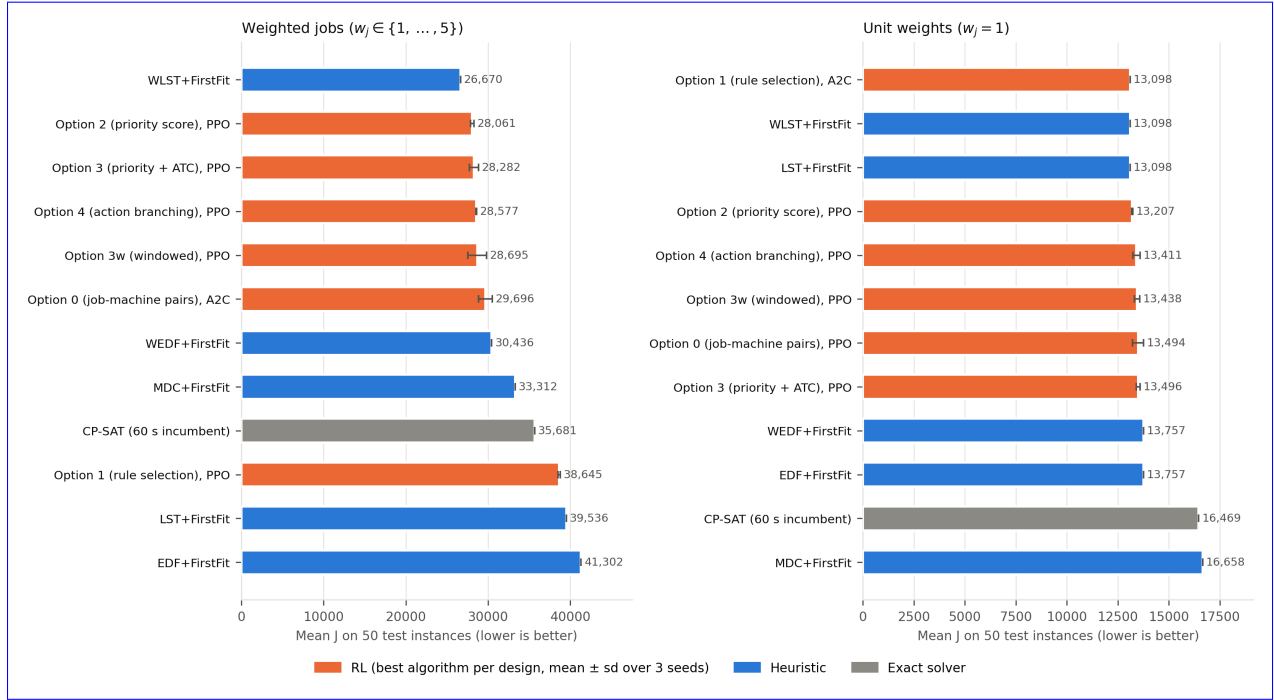


Figure 2: Offline results: the best final agent of each design (orange, mean $\pm$ s.d. over 3 seeds) against the leading heuristics (blue) and CP-SAT's 60-second schedule (grey). Left: weighted jobs; right: unit weights.

	Offline, weighted jobs	Offline, unit weights	Online, weighted jobs ($\rho = 0.95$)
J only	+5.2% heuristic	+0.0% tie	+13.3% heuristic
J + linear tardiness ($\times 1$)	-1.8% RL	+0.0% tie	+14.9% heuristic
J + linear tardiness ($\times 4$)	-10.3% RL	+0.0% tie	+14.4% heuristic
J + late jobs ($\times 1$)	-10.0% RL	+1.2% heuristic	+14.4% heuristic
J + late jobs ($\times 4$)	-1.4% RL	+10.8% heuristic	+11.5% heuristic
J + energy ($\times 1$)	+4.1% heuristic	+0.7% heuristic	+8.2% heuristic
All four terms ($\times 1$)	-8.6% RL	+2.1% heuristic	+11.3% heuristic

Best RL agent vs best heuristic on each objective. Negative (blue) = RL better; positive (red) = heuristic better.

Figure 3: When does RL win? Best RL agent against the best heuristic on each objective (rows) and setting (columns); blue means RL is better, red means the heuristic is better. Weights are multiples of the reference values, which make each added term equal to J on the best heuristic's schedule. Only RL agents with at least two evaluated seeds are included.

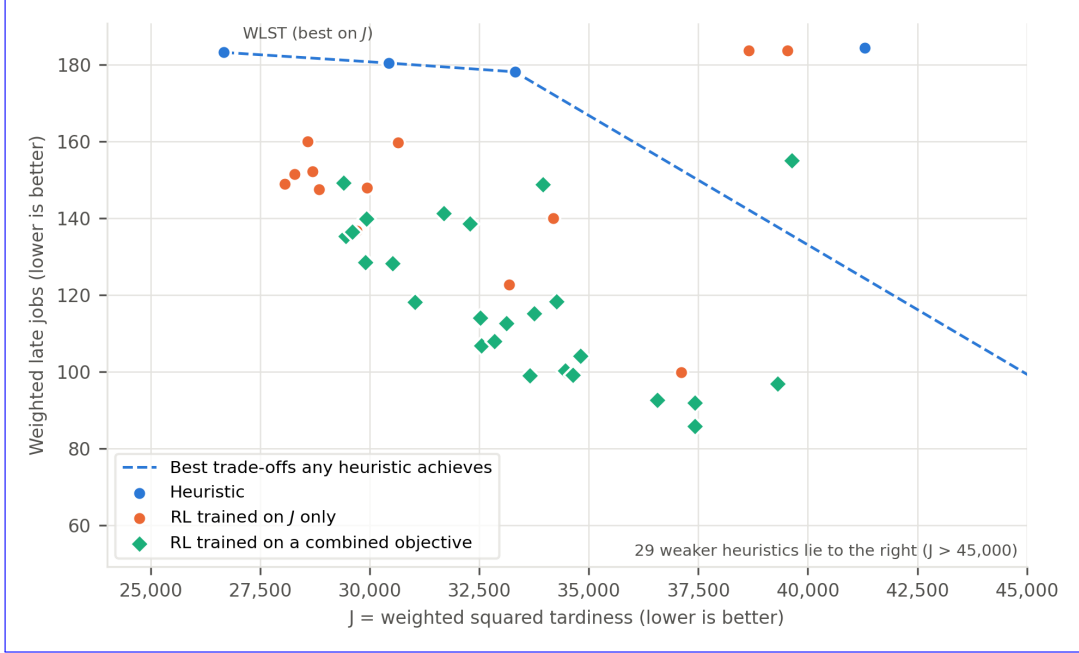


Figure 4: Offline, weighted jobs: J against the weighted number of late jobs (both lower is better) for every heuristic and every RL agent with at least two seeds. The dashed line joins the best trade-offs among the heuristics; points below it achieve combinations no heuristic reaches.

second seed of the same setup scored about 30,100, so this was not reproducible. Online A2C did not learn: Option 0 A2C produced the same schedule as FCFS+FirstFit ($J = 44,920$) on all three seeds.

Table 2: Online ($\rho = 0.95$) mean test J by training budget, PPO, mean $\pm$ s.d. over seeds (EDF+Consolidate: 21,986).

Design	300k steps	1M steps	2M steps
Option 0	49,404 $\pm$ 18,050	33,966 $\pm$ 3,699	–
Option 2	39,947 $\pm$ 11,949	26,515 $\pm$ 4,564	24,921 $\pm$ 2,555 (2 seeds)
Option 3	44,827 $\pm$ 2,890	30,611 $\pm$ 1,909	–

4.3 Other findings

Action-space design. Learning a job priority (Options 2 and 3) worked best in every setting. Picking job-machine pairs directly (Option 0) was 6% worse offline and still 55% behind online after 1M steps, and choosing job and machine with two heads (Option 4), competitive offline, failed online ($J = 139,721$). Rule selection (Option 1) mostly collapsed onto a single rule: with unit weights it reproduced LST exactly on all three seeds, and with weighted jobs its schedules had the same number of late jobs as LST (61.0) and a similar J (within 2.3% of LST).

Training stability. In 8 independent offline runs of the same Option 2 configuration (default hyperparameters, 300k steps), one diverged ($J = 69,367$ against 28,352–30,925 for the other seven). Online seeds differ much more: Option 2 at 300k steps ranged from 30,076 to 53,231.

Energy. RL lost to the best heuristic (WLST+Consolidate) on every J + energy objective, by 3.4–4.8%. Training with the energy term made no run diverge in 8 seeds, but it did not make training meaningfully more stable either: those runs were 5–25% worse on J , the expected cost of trading J for energy.

No single heuristic wins everywhere. The best rule changed with deadline tightness and load: EDF with loose offline deadlines ($TF = 0.2$, where it reaches $J = 0$), WLST with medium and tight deadlines, LST with unit weights, WLST+Consolidate online at $\rho = 0.50$ and 0.75 , EDF+Consolidate at $\rho = 0.95$, and MDC+Consolidate under overload ($\rho = 1.10$). Online, Consolidate placement was best at every load.

Hyperparameter tuning. The library defaults were the best configuration in 8 of the 24 studies, including both studies for Options 0 and 2 PPO online that were stopped early.

5 Discussion

When does RL beat the heuristics? Our results suggest three conditions. First, *the objective must not be one that a simple rule already solves well*. With unit weights and squared tardiness, LST matched every proven optimum on small instances, and RL could at best reproduce it. When the objective mixes several goals (Section 4.1), no single rule is designed for the combination, and RL won by up to 10%. Second, *jobs must differ in importance*. The offline multi-objective advantage disappeared with unit weights. Weights give a learned priority more to work with, since the right order now depends jointly on deadline, length and weight. This condition was tested offline only, because every online setting used weighted jobs. Third, *the agent must be trained enough for the setting*. Offline, 1M steps was enough to come within 5% of the best heuristic. Online, 300k steps left RL 70% behind, and 1–2M steps closed most of the gap for the best design.

RL and heuristic design. The best offline heuristic, WLST, did not exist at the start of the project: it was constructed after RL agents had beaten every published rule by 29%. One reading is that RL lost to WLST. A fairer reading is that RL found, automatically and within 5%, a level of performance that took a purpose-built rule to match. This suggests that RL could also serve as a tool for heuristic design, by revealing that better rules exist for a new objective. It also shows how much results depend on the choice of baselines. Studies that compare RL only with standard rules would have reported a clear RL win here.

Why the multi-objective gains need care. At moderate weights, the agents that won the composite objectives were trained on J alone. Their schedules happened to have fewer

late jobs and lower linear tardiness than WLST’s (for example, Option 0 A2C: 52.7 late jobs against 61.7; Fig. 4). WLST is tuned for squared tardiness and gives up the other metrics to get it. This part of the advantage is therefore a side effect of how RL schedules, not a trade-off it learned, and it is exactly the kind of pattern in non-optimised metrics that must not be over-interpreted. At larger weights, agents trained on the composite objective took over, which does indicate learning. Re-running these agents with the tuned settings and full 1M-step budget is in progress.

Why online is harder. Online returns depend heavily on the random arrival sequence, which the agent cannot predict, so the training signal is much noisier than offline [54]. Online episodes are also roughly ten times longer, and the observation has to cover over a thousand job slots. Both effects mean more samples are needed, consistent with the budget results. Option 0’s much larger action space compounds the problem [44].

Action spaces and energy. Priority learning worked best, in line with DeepRM and Decima, which also let the network choose the job and leave placement to the system [20, 21]. Rule selection collapsed onto one rule, so it can at best match the best rule in its menu. Options 2 and 3 always place with First-Fit and so cannot consolidate jobs onto fewer machines, a likely (still untested) reason they lose on energy.

Practical considerations. RL training is unstable (one diverged run in eight) and expensive, and tuning helped little: the defaults were best in 8 of 24 studies, consistent with the known sensitivity of RL to hyperparameters and the small tuning budgets typically used [69]. A heuristic needs no training and behaves the same every time. RL is therefore worth its cost mainly where the objective is complex and weighted, and where enough compute is available.

Limitations. All instances are synthetic and of one size (100 jobs, 10 machines), and results use 3 seeds. Picking the best of many RL variants on the test set slightly favours RL, although the best heuristic is picked the same way. CP-SAT did not reach optimality at 100 jobs, so the distance of RL and WLST from the optimum is unknown. The online 2M-step results rest on two seeds, and three online studies were stopped early.

6 Conclusion

This project formulated cloud resource allocation as a multi-resource scheduling problem and an MDP, built an offline and online simulation environment, and compared six RL action-space designs, trained with PPO and A2C under one uniform protocol, against 49 heuristics and an exact solver. On the main objective, weighted squared tardiness, RL came within 5% of the best heuristic offline and beat every published rule by 29%, but did not beat the best rule, which was purpose-built for that objective. RL did beat the best heuristic, by 2–10%, once linear tardiness or late jobs were added to the objective, provided jobs had different weights. Online, RL needed several times more training to approach the heuristics,

and it never won when energy was part of the objective. Learning a job priority was the most effective design. Getting RL to train reliably required careful diagnosis: reward scaling and event-driven idling were essential. In short, RL is most competitive when the objective combines goals that no single rule targets, jobs differ in importance, and training is sufficient. Where a simple rule is already near-optimal, the heuristic remains the better choice.

7 Future Work

- Extend the online study with longer training and more seeds, since the online gap was mainly a training-budget problem.
- Test generalisation to other deadline tightness and loads (in progress), larger clusters, and real traces such as Google’s [65] or Azure’s [66].
- Let RL choose placement as well as job order, e.g. with a consolidating placement rule, to address the energy gap.
- Train one agent conditioned on the objective weights [70, 71], and try graph neural network encoders [72].
- Use RL for heuristic discovery: extract interpretable rules from learned priorities.

Data Management Plan

All code, raw results (per-instance metrics for every method and seed), processed tables and figures, Optuna study logs and the chronological experiment log are in the public GitHub repository². Results are under **Results/** (raw runs, tuning studies, and the processed **ALL_RESULTS** archive), and every number in this report can be regenerated from them with the scripts provided. All instances are synthetic and generated from fixed random seeds, so no personal or sensitive data is involved. Trained model files are regenerated by the training scripts and are not version-controlled. The version cited in this report is a tagged release on the **main** branch.

Generative AI Use Statement

Claude Code (Anthropic) was used as a coding and research assistant. The original environment and the translation of the mathematical formulation into code were done by hand. Claude Code was used to implement models, training and evaluation, and to run experiments, under the following rules:

- **Context.** Claude was given the latest version of the report, formulations and experiment logs.

²<https://github.com/Ethan-Ty-Riekert/Neural-Knapsack-Research-Project>

- **Rules.** Every design decision had to be cited and justified, or derived, and implementation and discussion had to remain mathematically grounded.
- **Benchmarking and results.** Experiments were run by Claude from a queue, and every result was reviewed personally.
- **Diagnostics and research.** Unexpected results and failures were investigated and checked against the literature.
- **Recording.** Every decision and result was recorded in an experiment log.

All modelling decisions and formulations were reviewed and approved personally. Using Claude allowed far more experiments, analyses and model iterations than would otherwise have been possible, and made it easier to discuss modelling decisions against other papers. Separate Claude agents were also used weekly to check the code’s efficiency, mathematical rigour and factual accuracy. Claude was also used to help edit the text of this report; all content was reviewed and approved by the author.

References

- [1] K. Saidi and D. Bardou, “Task scheduling and vm placement to resource allocation in cloud computing: Challenges and opportunities,” *Cluster Computing*, vol. 26, no. 5, pp. 3069–3087, 2023. [Online]. Available: www.proquest.com/docview/2918267010
- [2] A. Mohamed, M. Hamdan, S. Khan, A. Abdelaziz, S. F. Babiker, M. Imran, and M. N. Marsono, “Software-defined networks for resource allocation in cloud computing: A survey,” *Computer Networks*, vol. 195, p. 108151, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S1389128621002152>
- [3] V. V. Vinothina, R. Sridaran, and P. Ganapathi, “A survey on resource allocation strategies in cloud computing,” *International Journal of Advanced Computer Science and Applications*, 2012. [Online]. Available: <https://doi.org/10.14569/IJACSA.2012>
- [4] S. M. Parikh, “A survey on cloud computing resource allocation techniques,” in *2013 Nirma University International Conference on Engineering (NUiCONE)*. IEEE, 2013, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/NUiCONE.2013.6780076>
- [5] M. H. Mohamaddiah, A. Abdullah, S. Subramaniam, and M. Hussin, “A survey on resource allocation and monitoring in cloud computing,” *International Journal of Machine Learning and Computing*, vol. 4, no. 1, pp. 31–38, 2014. [Online]. Available: <https://doi.org/10.7763/IJMLC.2014.V4.382>
- [6] N. Devi, S. Dalal, K. Solanki *et al.*, “A systematic literature review for load balancing and task scheduling techniques in cloud computing,” *Artificial Intelligence Review*, vol. 57, p. 276, 2024. [Online]. Available: <https://link.springer.com/article/10.1007/s10462-024-10925-w>
- [7] B. Wang, Z. Lin, W. Kong, and H. Dong, “Bin packing optimization via deep reinforcement learning,” *IEEE Robotics and Automation Letters*, vol. 10, no. 3, pp. 2542–2549, 2025.
- [8] N. Dahmani, A. Nazir, I. Taleb, and S. M. S. Bukhari, “Reinforcement learning based intelligent optimisation for bin packing problems: A review,” *Array*, vol. 28, p. 100616, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2590005625002437>
- [9] S. Alharthi, A. Alshamsi, A. Alseiari, and A. Alwarafy, “Auto-scaling techniques in cloud computing: Issues and research directions,” *Sensors*, vol. 24, no. 17, p. 5551, 2024. [Online]. Available: <https://www.mdpi.com/1424-8220/24/17/5551>
- [10] S. Bhatnagar and R. Mahant, “Dynamic resource allocation in cloud computing environments: Ai-driven approaches for optimizing workload distribution and cost efficiency,” in *Proceedings of International Conference on Paradigms of Communication, Computing and Data Analytics (PCCDA 2025)*, ser. Algorithms for Intelligent Systems, S. J. Nanda, H. Mittal, and M. H. Lim, Eds. Singapore: Springer, 2026. [Online]. Available: https://link.springer.com/chapter/10.1007/978-981-96-6843-4_23

- [11] H. Technologies, *Introduction to Cloud Computing Computing*. New York: Springer, 2022. [Online]. Available: <https://link.springer.com/book/10.1007/978-981-19-3026-3>
- [12] Q. Zhang, L. Cheng, and R. Boutaba, “Cloud computing: State-of-the-art and research challenges,” *Journal of Internet Services and Applications*, vol. 1, pp. 7–18, 2010. [Online]. Available: <https://link.springer.com/article/10.1007/s13174-010-0007-6>
- [13] Z. Y. Lee and X. Tang, “Evaluation of dynamic vector bin packing for virtual machine placement,” in *arXiv preprint*, 2026. [Online]. Available: <https://arxiv.org/html/2602.14704v1>
- [14] V. Anuradha and D. Sumathi, “A survey on resource allocation strategies in cloud computing,” *International Conference on Information Communication and Embedded Systems (ICICES2014)*, pp. 1–7, 2014.
- [15] R. Makkar and N. Mangla, “Load balancing and task scheduling methods in cloud computing – a review,” in *2024 7th International Conference on Contemporary Computing and Informatics (IC3I)*. IEEE, 2024, pp. 1037–1043.
- [16] R. Alkhalifa, F. Alkhomayes, B. Almazroua, D. Alhaidan, M. Alothman, and J. Almuheidib, “A comparative review of parallel exact, heuristic, metaheuristic, and hybrid optimization techniques for the traveling salesman problem,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.18278>
- [17] J. K. Lenstra, A. H. G. Rinnooy Kan, and P. Brucker, “Complexity of machine scheduling problems,” *Annals of Discrete Mathematics*, vol. 1, pp. 343–362, 1977.
- [18] A. Rathee and S. Dalal, “A systematic literature review of machine learning-based resource allocation techniques in cloud computing,” *Computing Archives for Informatics and Numerical Computation*, vol. 107, no. 9, p. 179, 09 2025. [Online]. Available: <https://www.proquest.com/scholarly-journals/systematic-literature-review-machine-learning/docview/3239203131/se-2>
- [19] J. Zhang, B. Zi, and X. Ge, “Attend2pack: Bin packing through deep reinforcement learning with attention,” *arXiv preprint arXiv:2107.04333*, 2021.
- [20] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*, 2016.
- [21] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM)*, 2019.
- [22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.

- [23] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *International Conference on Machine Learning (ICML)*, 2016.
- [24] V. R. Konda and J. N. Tsitsiklis, “Actor-critic algorithms,” in *Advances in Neural Information Processing Systems*, vol. 12. MIT Press, 1999, pp. 1008–1014. [Online]. Available: <https://proceedings.neurips.cc/paper/1999/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf>
- [25] M.-A. Chadia and H. Mousannifa, “Understanding reinforcement learning algorithms: The progress from basic q-learning to proximal policy optimization,” *arXiv preprint arXiv:2304.00026*, 2023. [Online]. Available: <https://arxiv.org/abs/2304.00026>
- [26] E. K. Burke, M. Gendreau, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, and R. Qu, “Hyper-heuristics: A survey of the state of the art,” *Journal of the Operational Research Society*, vol. 64, no. 12, pp. 1695–1724, 2013.
- [27] S. Lassoued, A. Gobachew, S. Lier, and A. Schwung, “Policy-based deep reinforcement learning hyperheuristics for job-shop scheduling problems,” *arXiv preprint arXiv:2601.11189*, 2026.
- [28] N. Marques, G. Figueira, and L. Guimarães, “Dynamic dispatching rule selection for the job shop scheduling problem,” *Computers and Industrial Engineering*, vol. 210, p. 111471, 2025.
- [29] B. Min and C. O. Kim, “State-dependent parameter tuning of the apparent tardiness cost dispatching rule using deep reinforcement learning,” *IEEE Access*, vol. 10, pp. 20 187–20 198, 2022.
- [30] X. Chen, R. Bai, R. Qu, J. Dong, and Y. Jin, “Deep reinforcement learning assisted genetic programming ensemble hyper-heuristics for dynamic scheduling of container port trucks,” *IEEE Transactions on Evolutionary Computation*, vol. 29, no. 4, 2024.
- [31] A. P. J. Vepsäläinen and T. E. Morton, “Priority rules for job shops with weighted tardiness costs,” *Management Science*, vol. 33, no. 8, pp. 1035–1047, 1987.
- [32] L. Perron and V. Furnon, “Or-tools,” Google. [Online]. Available: https://developers.google.com/optimization/cp/cp_solver
- [33] R. Anayat, “Cloud-based reinforcement learning in resource-constrained environments: Real-time performance optimization in autonomous systems,” 11 2024.
- [34] W. Yang, A. Acuto, Y. Zhou, and D. Wojtczak, “A survey for deep reinforcement learning based network intrusion detection,” *Applied AI Letters*, vol. 7, no. 2, p. 22, Jun 01 2026. [Online]. Available: <https://www.proquest.com/scholarly-journals/survey-deep-reinforcement-learning-based-network/docview/3314625396/se-2>
- [35] S. Huang and S. Ontañón, “A closer look at invalid action masking in policy gradient algorithms,” in *Proceedings of the International FLAIRS Conference*, 2020.

- [36] P. Ladosz, L. Weng, M. Kim, and H. Oh, “Exploration in deep reinforcement learning: A survey,” *Information Fusion*, vol. 85, pp. 1–22, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1566253522000288>
- [37] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, May 1992. [Online]. Available: <https://doi.org/10.1007/BF00992696>
- [38] J. Skalse, N. Howe, D. Krashenninnikov, and D. Krueger, “Defining and characterizing reward hacking,” *arXiv preprint arXiv:2209.13085*, 2022.
- [39] A. Beloglazov, J. Abawajy, and R. Buyya, “Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing,” *Future Generation Computer Systems*, vol. 28, no. 5, pp. 755–768, 2012. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X11000689?via%3Dihub>
- [40] X. Fan, W.-D. Weber, and L. A. Barroso, “Power provisioning for a warehouse-sized computer,” in *Proceedings of the 34th Annual International Symposium on Computer Architecture (ISCA)*, 2007, pp. 13–23. [Online]. Available: <https://www.barroso.org/publications/PowerProvisioning.pdf>
- [41] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley, 1975.
- [42] B. Giffler and G. L. Thompson, “Algorithms for solving production-scheduling problems,” *Operations Research*, vol. 8, no. 4, pp. 487–503, 1960.
- [43] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *International Conference on Machine Learning (ICML)*, 1999.
- [44] G. Dulac-Arnold, R. Evans, H. van Hasselt, P. Sunehag, T. Lillicrap, J. Hunt, T. Mann, T. Weber, T. Degris, and B. Coppin, “Deep reinforcement learning in large discrete action spaces,” arXiv, Tech. Rep. 1512.07679, 2015.
- [45] O. Vinyals, M. Fortunato, and N. Jaitly, “Pointer networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [46] W. Kool, H. van Hoof, and M. Welling, “Attention, learn to solve routing problems!” in *International Conference on Learning Representations (ICLR)*, 2019.
- [47] A. Tavakoli, F. Pardo, and P. Kormushev, “Action branching architectures for deep reinforcement learning,” arXiv, Tech. Rep. 1711.08946, 2018.
- [48] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [49] S. Huang, A. Kanervisto, A. Raffin, W. Wang, S. Ontañón, and R. F. J. Dossa, “A2C is a special case of PPO,” 2022.

- [50] C. Tessler, D. J. Mankowitz, and S. Mannor, “Reward constrained policy optimization,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [51] A. Stooke, J. Achiam, and P. Abbeel, “Responsive safety in reinforcement learning by pid lagrangian methods,” in *International Conference on Machine Learning (ICML)*, 2020.
- [52] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Madry, “Implementation matters in deep policy gradients: A case study on PPO and TRPO,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [53] M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Orsini, S. Girgin, R. Marinier, L. Hussenot, M. Geist, O. Pietquin, M. Michalski, S. Gelly, and O. Bachem, “What matters for on-policy deep actor-critic methods? a large-scale study,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [54] H. Mao, S. B. Venkatakrishnan, M. Schwarzkopf, and M. Alizadeh, “Variance reduction for reinforcement learning in input-driven environments,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [55] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel, “Asymmetric actor critic for image-based robot learning,” in *Robotics: Science and Systems (RSS)*, 2018.
- [56] M. L. Pinedo, *Scheduling: Theory, Algorithms, and Systems*, 5th ed. Springer, 2016.
- [57] R. L. Graham, “Bounds on multiprocessing timing anomalies,” *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969.
- [58] W. E. Smith, “Various optimizers for single-stage production,” *Naval Research Logistics Quarterly*, vol. 3, no. 1-2, pp. 59–66, 1956.
- [59] J. J. Kanet and X. Li, “A weighted modified due date rule for sequencing to minimize weighted tardiness,” *Journal of Scheduling*, vol. 7, no. 4, pp. 261–276, 2004.
- [60] D. C. Carroll, “Heuristic sequencing of single and multiple component jobs,” Ph.D. dissertation, Sloan School of Management, Massachusetts Institute of Technology, 1965.
- [61] D. S. Johnson, “Near-optimal bin packing algorithms,” *Journal of the ACM*, 1973.
- [62] R. Grandl, G. Ananthanarayanan, S. Kandula, S. Rao, and A. Akella, “Multi-resource packing for cluster schedulers,” *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 4, pp. 455–466, 2014.
- [63] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in *Proceedings of ICNN’95 – International Conference on Neural Networks*, vol. 4, 1995, pp. 1942–1948.
- [64] C. N. Potts and L. N. Van Wassenhove, “A branch and bound algorithm for the total weighted tardiness problem,” *Operations Research*, vol. 33, no. 2, pp. 363–377, 1985.

- [65] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, “Heterogeneity and dynamicity of clouds at scale: Google trace analysis,” in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2012.
- [66] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms,” in *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*, 2017.
- [67] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2019.
- [68] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in *Advances in Neural Information Processing Systems 24 (NeurIPS 2011)*, 2011, pp. 2546–2554.
- [69] T. Eimer, M. Lindauer, and R. Raileanu, “Hyperparameters in reinforcement learning and how to tune them,” arXiv, Tech. Rep. 2306.01324, 2023.
- [70] A. Abels, D. Roijers, T. Lenaerts, A. Nowé, and D. Steckelmacher, “Dynamic weights in multi-objective deep reinforcement learning,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 97, 2019, pp. 11–20.
- [71] C. F. Hayes, R. Rădulescu, E. Bargiacchi, J. Källström, M. Macfarlane, M. Raymond *et al.*, “A practical guide to multi-objective reinforcement learning and planning,” *Autonomous Agents and Multi-Agent Systems*, vol. 36, no. 1, p. 26, 2022.
- [72] “Graph neural networks for job shop scheduling problems: A survey,” *arXiv preprint arXiv:2406.14096*, 2024.

A Nomenclature

Table 3: Nomenclature

Sets	
J	Set of jobs, indexed by j
J_t	Jobs waiting (arrived, not yet started) at time t
P_t	Jobs processing (started, not finished) at time t
M	Set of machines, indexed by m
R	Set of resource types, indexed by r
$\mathcal{T}$	Set of discrete time periods, indexed by t
Parameters	
p_j	Processing duration of job j
a_{jr}	Requirement of job j for resource r
C_r	Capacity of each machine for resource r
H	Length of the planning horizon
d_j	Due date of job j
w_j	Priority weight of job j
τ_j	Arrival time of job j (0 offline)
ν, ρ	Arrival rate and load factor (online)
TF, RDD	Tardiness factor and due-date range (offline deadlines)
$\lambda_1, \lambda_2, \lambda_3$	Weights in the initial objective Eq. (3)
$\lambda_T, \lambda_U, \lambda_E$	Weights in the final objective Eq. (4)
Decision variables	
x_{jmt}	1 if job j starts on machine m at time t ; 0 otherwise
y_m	1 if machine m is used; 0 otherwise
s_j	Start time of job j
T_j	Tardiness of job j , $\max(0, s_j + p_j - d_j)$
U_j	1 if job j finishes after d_j ; 0 otherwise
E	Energy: total active machine-time
U_{mrt}	Total usage of resource r on machine m at time t
θ	Peak utilisation ratio, $\max_{m,r,t} U_{mrt}/C_r$

B Constraints

Let $\mathcal{T} = \{0, 1, \dots, \bar{H}\}$, where $\bar{H}$ is any upper bound on the time to finish all jobs (e.g. $H + \sum_j p_j$, under the extended horizon). For each job, let $\mathcal{T}_j = \{t \in \mathcal{T} : \tau_j \leq t \leq \bar{H} - p_j\}$ be its feasible start times.

Assignment. Each job starts exactly once, on exactly one machine, and never before it arrives:

$$\sum_{m \in M} \sum_{t \in \mathcal{T}_j} x_{jmt} = 1, \quad \forall j \in J. \quad (10)$$

Offline start rate. Offline, at most one job starts per time step:

$$\sum_{j \in J} \sum_{m \in M} x_{jmt} \leq 1, \quad \forall t \in \mathcal{T}. \quad (11)$$

Machine activation. A job can only be placed on an active machine:

$$x_{jmt} \leq y_m, \quad \forall j \in J, m \in M, t \in \mathcal{T}_j. \quad (12)$$

Start time.

$$s_j = \sum_{m \in M} \sum_{t \in \mathcal{T}_j} t x_{jmt}, \quad \forall j \in J. \quad (13)$$

Resource usage and capacity. A job started at t' occupies its resources during $[t', t' + p_j)$, so the usage at time t counts every job started within the previous p_j steps. Usage may never exceed capacity:

$$U_{mrt} = \sum_{j \in J} \sum_{\substack{t' \in \mathcal{T}_j \\ t - p_j < t' \leq t}} a_{jr} x_{jmt'} \leq C_r, \quad \forall m \in M, r \in R, t \in \mathcal{T}. \quad (14)$$

Tardiness, late jobs and peak utilisation.

$$T_j \geq s_j + p_j - d_j, \quad T_j \geq 0, \quad T_j \leq \bar{H} U_j, \quad \forall j \in J, \quad (15)$$

$$\theta \geq \frac{U_{mrt}}{C_r}, \quad \forall m \in M, r \in R, t \in \mathcal{T}. \quad (16)$$

Because the objectives are minimised and increase with T_j , U_j and θ , these inequalities hold with equality at the optimum. Finally, $x_{jmt}, y_m, U_j \in \{0, 1\}$.

C Hyperparameter Search Space

D Experimental Details

Offline deadlines. Following Potts and Van Wassenhove [64], each d_j is drawn uniformly from $[\bar{C}(1 - \text{TF} - \text{RDD}/2), \bar{C}(1 - \text{TF} + \text{RDD}/2)]$ and is at least p_j . Here $\bar{C}$ is a lower bound on the time to finish all jobs (the larger of the number of jobs, since one job starts per step, and the total work divided by the cluster's capacity), TF is the tardiness factor and $\text{RDD} = 0.6$ the range of due dates, which sets how spread out deadlines are.

Table 4: Hyperparameters searched by Optuna (ranges after [53]). “log” means sampled log-uniformly; lists are categorical choices. Trial 0 of every study used the library defaults instead.

Hyperparameter	PPO	A2C
Learning rate	$3 \times 10^{-5} - 10^{-3}$ (log)	$10^{-4} - 3 \times 10^{-3}$ (log)
Rollout length (all 4 environments)	2048, 4096, 8192, 16384	20, 80, 320
Minibatch size	64, 256, 1024	–
Epochs per update	3, 5, 10	–
Discount factor γ	0.99, 0.995, 0.999	0.99, 0.995, 0.999
GAE λ	0.9, 0.95, 0.98	0.9, 0.95, 1.0
Clip range	0.1, 0.2, 0.3	–
Entropy coefficient	$10^{-4} - 3 \times 10^{-2}$ (log)	$10^{-4} - 3 \times 10^{-2}$ (log)

Online deadlines and job sizes. A job’s deadline is its arrival time plus a slack drawn uniformly from $\{10, \dots, 60\}$ time steps ($\{5, \dots, 25\}$ in a tight-deadline variant). Durations and demands are log-normal, not Gaussian: the logarithm of the size is normally distributed (with $\sigma = 1$), giving many small jobs and a few large ones. The location parameter is set so that the mean equals that of the offline uniform distribution, and sizes are capped at four times the offline maximum (durations of at most 40 steps).

Tuning protocol. Each of the 24 studies ran 12 trials of 300,000 steps with 4 parallel environments; trial 0 used the Stable-Baselines3 defaults, which lie outside the log-scaled search space (the default entropy coefficient is 0). Every 75,000 steps, a trial reported its mean J on 5 validation instances. Optuna’s median pruner stopped the trial if this was worse than the median of the values earlier trials had reported at the same step; pruning only starts after 5 trials have finished. A completed trial’s score is its mean J on all 20 validation instances, and the best score gives the study’s single chosen configuration. The offline configuration was reused for the unit-weight setting. Three online PPO studies (Options 0, 2 and 4) trained at about 20 steps per second, roughly 4–5 hours per trial, and were stopped after 3–7 trials to meet the project deadline; the best completed trial was used as usual. The defaults were the best configuration in 8 of the 24 studies.

Faults fixed in the final training version. Each fault was measured directly in training logs.

1. *The critic did not learn.* Squared-tardiness returns are in the hundreds to thousands, and the value and policy losses share one clipped gradient, so the value error swamped each update. The critic explained none of the variance in returns, and A2C policies stayed close to uniformly random. Rewards are now divided by a running estimate of the standard deviation of the return [52, 53]; afterwards the critic explained 30–99.9% of the variance.
2. *Deterministic evaluation idled.* Idle is a single action, while the probability of starting a job is split across many (job, machine) actions. For an undecided policy, idle could

be the single most likely action in every state, so a “most likely action” rule idled indefinitely even though the trained (sampling) policy placed jobs 98% of the time. The two-stage rule of Section 3.7 fixes this, and is identical to the most-likely-action rule for confident policies.

3. *Idling could repeat forever.* An idle used to advance time by one step and return the agent to an unchanged decision. Event-driven idling advances to the next arrival or completion instead. Waiting between events never helps, since a job that fits at an intermediate step also fits now and finishes earlier, so no schedule is lost.

Together, these changes reduced offline Option 0 A2C’s validation J from 234,116 to 43,164.